



WEB SERVICES

I SERVIZI REST

WEB SERVICES

I principi dell'architettura REST

I principi dell'architettura REST

- Volendo essere formali, REST definisce un insieme di principi architetturali per la progettazione di un sistema.
- Rappresenta uno stile architeturale, cioè non si riferisce ad un sistema concreto e ben definito né si tratta di uno standard stabilito da un organismo di standardizzazione.
- La sua definizione è apparsa per la prima volta nel 2000 nella tesi di Roy Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, discussa presso l'Università della California, ad Irvine.
- In questa tesi si analizzavano alcuni principi alla base di diverse architetture software, tra cui appunto i principi di un'architettura software che consentisse di vedere il Web come una piattaforma per l'elaborazione distribuita.
- È bene precisare che i principi REST non sono necessariamente legati al Web, nel senso che si tratta di principi astratti di cui il World Wide Web ne risulta essere un esempio concreto.
- All'epoca questa visione del Web passò un po' in sordina, ma negli ultimi anni l'approccio REST è venuto alla ribalta come metodo per la realizzazione di Web Service altamente efficienti e scalabili ed ha al suo attivo un significativo numero di applicazioni.

I principi dell'architettura REST

- Il motivo è semplice: dal momento che il Web ha tutto quello che serve per essere considerata una piattaforma di elaborazione distribuita secondo i principi REST, non sono necessarie altre sovrastrutture per realizzare quello che è il Web programmabile.
- Il che è una dichiarazione di aperto antagonismo ai Web Service basati su SOAP.

Che cos'è REST?

- Abbiamo detto che REST non è un'architettura né uno standard, ma un insieme di linee guida per la realizzazione di una “architettura di sistema”. Ma quali sono questi principi che rendono il Web adatto a realizzare Web Service secondo l'approccio REST? Il tutto può essere riassunto nei seguenti cinque principi:
 - Identificazione delle risorse
 - Utilizzo esplicito dei metodi HTTP
 - Risorse autodescrittive
 - Collegamenti tra risorse
 - Comunicazione senza stato
- Questi principi rappresentano in realtà concetti ben noti perché ormai insiti nel Web che conosciamo. Li analizzeremo tuttavia sotto una prospettiva diversa, quella della realizzazione di Web Service.

Identificazione delle risorse

- Le risorse sono gli elementi fondamentali su cui si basano i Web Service RESTful, a differenza dei Web Service SOAP-oriented che sono basati sul concetto di chiamata remota.
- Per risorsa si intende un qualsiasi elemento oggetto di elaborazione.
- Per fare qualche esempio concreto, una risorsa può essere un cliente, un libro, un articolo, un qualsiasi oggetto su cui è possibile effettuare operazioni.
- Per fare un parallelo con la programmazione ad oggetti possiamo dire che una risorsa può essere assimilata ad una istanza di una classe.
- Il principio che stiamo analizzando stabilisce che ciascuna risorsa deve essere identificata univocamente. Essendo in ambito Web, il meccanismo più naturale per individuare una risorsa è dato dal concetto di URI.
- Il principale beneficio nell'adottare lo schema URI per identificare le risorse consiste nel fatto che esiste già, è ben definito e collaudato e non occorre pertanto inventarsene uno nuovo.

Identificazione delle risorse

- I seguenti sono esempi di possibili identificatori di risorse:
 - <http://www.myapp.com/clienti/1234>
 - <http://www.myapp.com/ordini/2011/98765>
 - <http://www.myapp.com/prodotti/7654>
 - <http://www.myapp.com/ordini/2011>
 - <http://www.myapp.com/prodotti?colore=rosso>
- Gli URI sono abbastanza autoesplicativi: i primi tre identificano rispettivamente un determinato cliente, un ordine ed un prodotto. Il quarto URI identifica l'insieme degli ordini del 2011, mentre l'ultimo URI identifica l'insieme dei prodotti di colore rosso.
- L'interpretazione che abbiamo dato a questi URI è però desunta dalla semantica delle parole contenute nelle sue parti.
- Dal punto di vista di un Web Service un URI è soltanto una stringa che identifica una risorsa, per cui anche <http://www.myapp.com/tgw34/2099ww> può essere un URI valido per identificare un cliente.

Mappare le azioni 'CRUD' sui metodi HTTP

- Una volta spiegato come individuare una risorsa abbiamo bisogno di un meccanismo per indicare quali operazioni effettuare su di esse.
- Il principio dell'uso esplicito dei metodi HTTP ci indica di sfruttare i metodi (o verbi) predefiniti di questo protocollo, e cioè GET, POST, PUT e DELETE.
- Facciamo un semplice esempio per provare a chiarire questo concetto:
 - Quando inseriamo un URI nella barra degli indirizzi di un browser stiamo in realtà chiedendo al browser di eseguire un metodo HTTP sulla risorsa individuata dall'URI.
 - Il metodo che implicitamente stiamo eseguendo è GET, il cui effetto è l'accesso ad una rappresentazione della risorsa identificata dall'URI.
- Dal punto di vista del codice, non avremo bisogno di un metodo del tipo `getClient(1234)` per ottenere la rappresentazione del cliente con codice 1234, sarà sufficiente sfruttare il metodo standard GET del protocollo HTTP sull'URI che identifica quel determinato cliente.

Mappare le azioni 'CRUD' sui metodi HTTP

- Questo rende uniforme l'invocazione di operazioni sulle risorse, cioè il client non ha bisogno di sapere qual è la specifica interfaccia da utilizzare per invocare il metodo che consente di ottenere la rappresentazione di un cliente.
- In un contesto non RESTful potremmo avere metodi come getClient() o getCustomer() o altra specifica dipendente dalle scelte di chi ha sviluppato il Web Service.
- In un contesto RESTful sappiamo già a priori come ottenere la rappresentazione di una risorsa. In altre parole, questo principio REST stabilisce una mappatura uno a uno tra le tipiche operazioni CRUD (creazione, lettura, aggiornamento, eliminazione di una risorsa) e i metodi HTTP.

Metodo HTTP	Operazione CRUD	Descrizione
POST	Create	Crea una nuova risorsa
GET	Read	Ottiene una risorsa esistente
PUT	Update	Aggiorna una risorsa o ne modifica lo stato
DELETE	Delete	Elimina una risorsa

Mappare le azioni 'CRUD' sui metodi HTTP

- È opportuno notare che questo principio è in contrasto con quella che è la tendenza generale nell'utilizzo dei metodi HTTP.
- Infatti, molto spesso viene utilizzato il metodo GET per eseguire qualsiasi tipo di interazione con il server.
- Ad esempio, spesso per l'inserimento di un nuovo cliente nell'ambito di un'applicazione Web viene eseguita una richiesta di tipo GET su un URI del tipo: <http://www.myapp.com/addCustomer?name=Rossi>
- Questo approccio non è conforme ai principi REST perchè il metodo GET serve per accedere alla rappresentazione di una risorsa e non per crearne una nuova.
- In pratica questo uso del metodo GET introduce un effetto collaterale che, tra l'altro, può avere delle conseguenze indesiderate se, ad esempio, l'URI viene invocato più volte o se la risorsa viene memorizzata in uno dei vari livelli di cache esistenti tra client e server.
- Come principio generale, nel progettare un Web Service in modalità REST è utile evitare l'uso di verbi negli URI ma limitarsi ad utilizzare nomi, ricordandosi che un URI identifica una risorsa.
- Allo stesso modo è opportuno sottolineare che il corpo di una richiesta HTTP con metodi PUT e POST è pensato per il trasferimento della rappresentazione di una risorsa e non per eseguire chiamate remote o altre attività simili.

WEB SERVICES

Stateless, autodefinizione
e
collegamenti tra risorse

Risorse autodescrittive

- Le risorse di per sè sono concettualmente separate dalle rappresentazioni restituite al client.
- Ad esempio, un Web Service non invia al client direttamente un record del suo database, ma una sua rappresentazione in una codifica dipendente dalla richiesta del client e/o dall'implementazione del servizio.
- I principi REST non pongono nessun vincolo sulle modalità di rappresentazione di una risorsa.
- Virtualmente possiamo utilizzare il formato che preferiamo senza essere obbligati a seguire uno standard.
- Di fatto, però, è opportuno utilizzare formati il più possibile standard in modo da semplificare l'interazione con i client.
- Inoltre, sarebbe opportuno prevedere rappresentazioni multiple di una risorsa, per soddisfare client di tipo diverso.
- Il tipo di rappresentazione inviata dal Web Service al client è indicato nella stessa risposta HTTP tramite un tipo MIME, così come avviene nella classica comunicazione tra Web server e browser.

Risorse autodescrittive

- Un client a sua volta ha la possibilità di richiedere una risorsa in uno specifico formato sfruttando l'attributo Accept di una richiesta HTTP di tipo GET, come mostrato nel seguente esempio:
 - `GET /clienti/1234 HTTP/1.1 Host: www.myapp.com Accept: application/vnd.myapp.cliente+xml`
- In questo caso il client richiede la rappresentazione del cliente con id 1234, in formato XML secondo lo schema definito da `vnd.myapp.cliente`.
- Se il Web Service lo supporta, la stessa risorsa può essere richiesta in formati diversi, come ad esempio JSON o HTML.
- La possibilità di rappresentazioni multiple produce alcuni benefici pratici: ad esempio, se abbiamo un output sia HTML, sia XML, possiamo consumare il servizio sia con un'applicazione sia con un comune browser.
- In altre parole, seguendo i principi REST nel progettare un'applicazione Web è possibile costruire sia una Web API che una Web UI.

Collegamenti tra risorse

- Un altro vincolo dei principi REST consiste nella necessità che le risorse siano tra loro messe in relazione tramite link ipertestuali. Questo principio è anche noto come HATEOAS, dall'acronimo di Hypermedia As The Engine Of Application State, e pone l'accento sulle modalità di gestione dello stato dell'applicazione.
- In sostanza, tutto quello che un client deve sapere su una risorsa e sulle risorse ad essa correlate deve essere contenuto nella sua rappresentazione o deve essere accessibile tramite collegamenti ipertestuali.
- Ad esempio, la rappresentazione di un ordine in un linguaggio XML-based deve contenere gli eventuali collegamenti agli articoli ed al cliente correlati:

```
<ordine>  
  <numero>12345678</numero>  
  <data>01/07/2011</data>  
  <cliente rif="http://www.myapp.com/clienti/1234" />  
  <articoli> <articolo rif="http://www.myapp.com/prodotti/98765" />  
    <articolo rif="http://www.myapp.com/prodotti/43210" /> </articoli>  
</ordine>
```

Collegamenti tra risorse

- In questo modo il client può accedere alle risorse correlate seguendo semplicemente i collegamenti contenuti nella rappresentazione della risorsa corrente.
- Il fatto di utilizzare un URI come identificatore di una risorsa, quindi un meccanismo standard e consolidato, consente al client di accedere anche a risorse messe a disposizione da altre applicazioni che girano eventualmente su altri server.
- Inoltre, la possibilità di avere collegamenti ipertestuali all'interno della rappresentazione di una risorsa rappresenta un meccanismo per poter gestire lo stato dell'applicazione, come vedremo in seguito.

Comunicazione senza stato

- Il principio della comunicazione stateless è ben noto a chi lavora con il Web.
- Questa è infatti una delle caratteristiche principali del protocollo HTTP, cioè ciascuna richiesta non ha alcuna relazione con le richieste precedenti e successive.
- Lo stesso principio si applica ad un Web Service RESTful, cioè le interazioni tra client e server devono essere senza stato.
- È importante sottolineare che sebbene REST preveda la comunicazione stateless, non vuol dire che un'applicazione non deve avere stato.
- La responsabilità della gestione dello stato dell'applicazione non deve essere conferita al server, ma rientra nei compiti del client.
- La principale ragione di questa scelta è la scalabilità: mantenere lo stato di una sessione ha un costo in termini di risorse sul server e all'aumentare del numero di client tale costo può diventare insostenibile.
- Inoltre, con una comunicazione senza stato è possibile creare cluster di server che possono rispondere ai client senza vincoli sulla sessione corrente, ottimizzando le prestazioni globali dell'applicazione.

WEB SERVICES

Consumare RESTful Web Service

Creare un client RESTful

- I cinque principi analizzati nelle lezioni precedenti ci consentono di definire un Web Service aderente ad una architettura in stile REST. Ma cosa vuol dire questo in pratica?
- Iniziamo concentrandoci nell'interazione con un Web Service esistente.
- Creeremo un client RESTful, cioè in grado interagire con un Web Service di tipo REST, in questo modo metteremo in pratica quanto appreso sui principi REST. Ecco ciò di cui abbiamo bisogno:
 - un client HTTP
 - le URI delle risorse a cui vogliamo accedere
 - la capacità di interpretare il formato delle rappresentazione delle risorse
- Creare un client RESTful è in un certo senso simile a realizzare un piccolo browser Web specializzato nell'interpretazione delle risorse messe a disposizione dal Web Service.

Consumare le API

- Scendendo nel concreto, prendiamo il servizio Web RESTful esposto dalle Blogger API di Google, che consente di interagire con la nota piattaforma di blogging.
- Grazie a queste API un'applicazione può accedere ai post di un blog, può crearne di nuovi e modificare o eliminare quelli esistenti.
- Seguendo le specifiche delle API (https://developers.google.com/blogger/docs/2.0/developers_guide_protocol?csw=1) proveremo a creare un semplice client per la gestione dei post di un ipotetico blog.
- Si tratta essenzialmente di un esercizio a scopo didattico, per cui il codice che mostreremo non ha la pretesa di essere completo né affidabile.
- Ignoreremo ad esempio la necessità di un'autenticazione e tutti gli aspetti di sicurezza che verranno affrontati più avanti.

Consumare le API

- Come prima cosa abbiamo bisogno di un client HTTP.
- Potremmo pensare ad un classico browser Web, cosa da non trascurare come forma rudimentale di debugger, ma parlando della realizzazione di un client non possiamo prescindere da quello che lo specifico linguaggio e/o ambiente di programmazione ci mette a disposizione per gestire il protocollo HTTP.
- Non dovrebbe essere particolarmente complesso riportare le operazioni che vedremo in un altro ambiente di programmazione.
- Evitiamo volutamente l'utilizzo di framework ed approcci a più alto livello perchè il nostro obiettivo in questa sede è di concentrarci sui concetti di base dell'approccio REST.

Consumare le API

- La seconda cosa di cui abbiamo bisogno è l'insieme delle URI che individuano le risorse che andremo a gestire. Seguendo le specifiche delle Blogger API e supponendo che il nostro profileID sia 1234567890, possiamo riepilogare le risorse accessibili nel seguente modo:

URI (con prefisso <code>http://www.blogger.com/feeds/</code>)	Descrizione
<code>/1234567890/blogs</code>	Elenco dei blog del nostro profilo
<code>/1234567890/blogs/12345</code>	Blog con ID 12345
<code>/12345/posts/default</code>	Elenco dei post del blog 12345
<code>/12345/posts/default/67890</code>	Post con ID 67890 del blog 12345
<code>/12345/posts/default?published-min=2011-01-01T00:00:00&published-max=2011-06-30T23:59:59</code>	Elenco dei post del blog 12345 pubblicati dal primo gennaio 2011 al 30 giugno 2011

Consumare le API

- La terza ed ultima cosa che ci occorre sapere è il formato con cui verranno rappresentate le risorse richieste. Nel nostro caso, il formato previsto dalle Blogger API è Atom 1.0.

Consumare le API

- Una volta stabilite le URI che identificano le risorse non ci resta vedere come ottenerne una sfruttando il metodo GET.
- Supponendo quindi di voler richiedere l'elenco dei post del blog con identificatore 12345:
 - <http://www.blogger.com/feeds/1234567890/blogs/12345>
- In sintesi, creiamo una richiesta verso l'URI che identifica l'elenco dei post del nostro blog, specifichiamo il metodo GET, inviamo la richiesta e rimaniamo in attesa della risposta.
- Se lo stato della risposta è OK, accediamo al contenuto della risposta e lo interpretiamo tramite il codice di gestione che dovrà essere implementato per recuperare le informazioni di interesse
- La rappresentazione della risorsa richiesta sarà qualcosa di simile a quanto segue:

Consumare le API

```
<?xml version='1.0' encoding='utf-8'?>
<?xml-stylesheet href="http://www.blogger.com/styles/atom.css" type="text/css"?>
<feed xmlns='http://www.w3.org/2005/Atom' xmlns:gd='http://schemas.google.com/g/2005' gd:etag='W/"D08FQn8-eip7ImA9WxZbFEw."'>
  <id>tag:blogger.com,1999:blog-12345</id>
  <updated>2011-04-17T00:03:33.152-07:00</updated>
  <title>Il mio blog</title>
  <subtitle type='html'></subtitle>
  <link rel='http://schemas.google.com/g/2005#feed' type='application/atom+xml' href='http://ilmioblog.blogspot.com/feeds/posts/default' />
  <link rel='self' type='application/atom+xml' href='http://www.blogger.com/feeds/12345/posts/default' />
  <link rel='alternate' type='text/html' href='http://ilmioblog.blogspot.com/' />
  <author>
    <name>Mario Rossi</name>
    <uri>http://www.blogger.com/profile/1234567890</uri>
    <email>noreply@blogger.com</email>
  </author>
  <generator version='7.00' uri='http://www2.blogger.com'>Blogger</generator>
  <entry gd:etag='W/"D0YHRn84eip7ImA9WxZUFk8."'>
    <id>tag:blogger.com,1999:blog-12345.post-67890</id>
    <published>2011-04-07T20:25:00.005-07:00</published>
    <updated>2011-04-07T20:25:37.132-07:00</updated>
    <title>Il mio primo post</title>
    <content type='html'>Questo È il mio primo post per il mio nuovo blog</content>
    <link rel='edit' type='application/atom+xml' href='http://www.blogger.com/feeds/12345/posts/default/67890' />
    <link rel='self' type='application/atom+xml' href='http://www.blogger.com/feeds/12345/posts/default/67890' />
    <link rel='alternate' type='text/html' href='http://ilmioblog.blogspot.com/2011/04/ilmioprimopost.html' />
    <author>
      <name>Mario Rossi</name>
      <uri>http://www.blogger.com/profile/1234567890</uri>
      <email>noreply@blogger.com</email>
    </author>
  </entry>
</feed>
```

Consumare le API

- In questo feed Atom si trova la rappresentazione dell'elenco dei post del nostro blog che nel caso specifico si tratta di un unico post.
- Il compito della procedura che decodifica la risposta consiste nell'interpretare questa rappresentazione e trasformarla in un oggetto facilmente manipolabile da parte del client, come potrebbe essere ad esempio il risultato di una deserializzazione.
- Come possiamo vedere, all'interno della rappresentazione dell'elenco dei post si trova l'URI dell'unico post
 - <http://www.blogger.com/feeds/12345/posts/default/67890>
- e l'URI del profilo dell'autore
- <http://www.blogger.com/profile/1234567890>
- Si tratta delle risorse collegate alla risorsa corrente. Quindi se la nostra applicazione avrà bisogno di accedere a queste risorse sarà sufficiente utilizzare i corrispondenti URI.

Consumare le API

- Le specifiche delle Blogger API prevedono la possibilità di selezionare un sottoinsieme dei post di un blog specificando dei criteri come ad esempio le date di pubblicazione.
- Ad esempio, se vogliamo selezionare i post pubblicati nella prima metà del 2011 possiamo eseguire una GET sul seguente URI:
 - <http://www.blogger.com/feeds/12345/posts/default?published-min=2011-01-01T00:00:00&published-max=2011-06-30T23:59:59>
- A prima vista questo modo di indicare una risorsa potrebbe apparire in contrasto con i principi RESTful.
- La presenza di parametri nell'URI potrebbe far pensare ad una chiamata di procedura piuttosto che ad un identificatore di risorsa.
- In realtà l'approccio è del tutto legittimo in quanto la stringa è univoca ed identifica esattamente la risorsa che vogliamo ottenere senza alcun effetto collaterale.
- La presenza dei parametri ci dà la sensazione di una chiamata di procedura (e con ogni probabilità l'implementazione sottostante lo è) ma da un punto di vista REST non si tratta che di una sequenza di caratteri per l'identificazione di una risorsa.

Creare una nuova risorsa

- Per la creazione di una nuova risorsa utilizziamo il metodo POST inviando al server la relativa rappresentazione in Atom, come quello mostrato nell'esempio:

```
<entry xmlns='http://www.w3.org/2005/Atom'>
  <title type='text'>Secondo post</title>
  <content type='xhtml'>
    <div xmlns="http://www.w3.org/1999/xhtml">
      <p>Questo è il mio secondo post!</p>
    </div>
  </content>
  <category scheme="http://www.blogger.com/atom/ns#" term="post" />
  <category scheme="http://www.blogger.com/atom/ns#" term="articoli" />
</entry>
```

- E inviamo la nostra richiesta di creazione di un nuovo post al server

Creare una nuova risorsa

- In pratica abbiamo creato una richiesta di tipo POST verso l'URI relativa all'elenco dei post del nostro blog, abbiamo specificato il tipo di rappresentazione della risorsa tramite il ContentType della richiesta, abbiamo associato allo stream delle richiesta il contenuto del nuovo post (variabile newPost) ed infine abbiamo inviato la richiesta al server.
- Se il codice di stato restituito dal server non corrisponde a Created (codice di stato HTTP 201), allora occorre gestire l'errore, altrimenti la creazione della risorsa ha avuto esito positivo.
- Nel caso specifico delle Blogger API, il server restituisce come contenuto la risorsa appena creata.
- Essa è sostanzialmente identica a quella che abbiamo inviato, anche se presenta qualche informazione in più, come ad esempio un elemento <id> che indica l'identificatore assegnato al post appena creato.

Modifica di una risorsa

- La modifica di una risorsa implica l'utilizzo del metodo PUT.
- Quindi, supponendo di voler modificare il post del nostro blog inviamo un'isola dati simile a quella precedentemente usata per la creazione.
- Ovviamente le uniche differenze riguardano l'URI della specifica risorsa che vogliamo modificare, che contiene anche l'id della risorsa e l'utilizzo del metodo PUT.

Eliminazione di una risorsa

- Concludiamo con l'eliminazione di un post dal blog tramite il metodo DELETE.
- Come per la richiesta di una risorsa, ci limitiamo ad indicare l'URI del post da eliminare ed il metodo DELETE.

WEB SERVICES

Progettare ed implementare
servizi RESTful

Risorse e relazioni nei servizi RESTFul

- Dopo aver visto come consumare un Web Service in modalità REST, vediamo come è possibile realizzarne uno. In questa sezione affronteremo infatti le problematiche di progettazione di un Web Service che risponda ai principi REST e vedremo una semplice implementazione.
- Il Web Service che andremo a realizzare riguarda un classico negozio online a cui un client può sottoporre un ordine, interrogare o modificarne lo stato, richiedere fattura, ecc.
- Naturalmente semplificheremo l'intero processo per concentrarci sugli aspetti rilevanti per gli obiettivi di questa guida.
- Nella progettazione di un Web Service RESTful possiamo seguire alcune linee guida che ci permettono di ragionare su quello che vogliamo ottenere e come ottenerlo secondo i principi REST.
- Questi principi, infatti, devono essere sempre presenti in ogni nostra decisione nella fase di progettazione.
- Mostreremo le linee guida da seguire come una serie di passi durante i quali concentreremo la nostra attenzione su alcuni aspetti da definire per giungere al progetto finale pronto per l'implementazione.

Individuare le risorse

- Il primo passo da fare consiste nell'individuare le risorse da esporre tramite il Web service.
- È importante evidenziare che in questa fase dobbiamo pensare a come le risorse vengono viste da un ipotetico client e non a come vengono implementate internamente.
- Occorre infatti separare l'interfaccia verso l'esterno dai dettagli implementativi.
- Ad esempio, se creiamo un Web Service che si interfaccia ad una applicazione esistente, non tutte le proprietà di un oggetto del dominio dell'applicazione vanno rese disponibili.
- Il Web Service deve consentire l'accesso alle sole caratteristiche rilevanti ai fini dell'esecuzione delle richieste remote.
- Inoltre, non tutte le risorse del dominio dell'applicazione sono da rendere accessibili all'esterno.
- Ancora una volta dobbiamo limitarci a quelle risorse che sono rilevanti per il servizio da realizzare.

Individuare le risorse

- In sostanza, le risorse da esporre tramite un Web Service non sono una semplice mappatura del modello ad oggetti interno all'applicazione.
- Gli oggetti vanno reinterpretati alla luce degli scopi del Web Service da sviluppare.
- Ad esempio, nel progetto che andremo idealmente a realizzare vogliamo consentire agli utenti di un ipotetico negozio online di selezionare articoli, di fare ordini e di ottenere le relative fatture.
- Anche se l'applicazione sottostante gestisce clienti, fornitori ed altri oggetti, non metteremo questi oggetti tra le risorse da esporre tramite Web Service.
- Inoltre, sulle risorse che metteremo a disposizione limiteremo sia le caratteristiche accessibili sia le operazioni ammissibili.

Rappresentazione, URI e metodi di un web service RESTful

- Proseguiamo con la nostra progettazione con la definizione degli URI che identificheranno le nostre risorse.
- In linea di massima, dal momento che il numero effettivo delle risorse non è definito a priori e nella maggior parte dei casi è potenzialmente infinito, vengono definiti dei modelli di URI per l'individuazione delle risorse.
- Ad esempio, nel nostro caso un ordine con identificatore 123 può essere individuato da un URI della forma `http://www.mionegozio.com/ordini/123` oppure `http://www.mionegozio.com/ordini/?id=123`.
- Non esiste una regola per determinare se un URI è migliore di un altro.
- Un URI è solo una stringa per individuare una risorsa. È utile comunque seguire alcune indicazioni per definire gli URI del proprio Web Service:

Rappresentazione, URI e metodi di un web service RESTful

Raccomandazione	Descrizione
preferire l'utilizzo di nomi a quello di verbi	un URI del tipo <code>http://www.mionegozio.com/gestisciOrdini/123</code> , anche se a tutti gli effetti valido per individuare una risorsa, è un pò fuorviante perchè induce a pensare che ad esso è associata un'azione invece che una risorsa; ricordiamoci che in ambito REST gli URI individuano risorse e che le azioni sono demandate ai metodi HTTP
contenere la lunghezza degli URI	si tratta di una indicazione pratica che favorisce la leggibilità delle specifiche, ma anche di un limite fisico mposto dalla maggior parte dei Web server
preferire uno schema posizionale	in linea di massima è da preferire un modello di URI che sfrutta la struttura gerarchica di un percorso piuttosto che la presenza di più argomenti; ad esempio, se vogliamo individuare gli ordini in base alla data, è da preferire lo schema <code>http://www.mionegozio.com/ordini/2011/07/01</code> al posto di <code>http://www.mionegozio.com/ordini/?anno=2001&mese=07&giorno=01</code>
evitare le estensioni	è bene tener presente che gli URI introducono un vincolo di accoppiamento con il client, quindi occorre fare molta attenzione nella loro definizione in quanto eventuali modifiche successive del modello hanno conseguenze sul client. In questa ottica sono da evitare le estensioni che legano un Web Service alla sua implementazione. Ad esempio è da preferire un URI del tipo <code>http://www.mionegozio.com/ordini/?id=123</code> ad uno del tipo <code>http://www.mionegozio.com/ordini/ordine.php?id=123</code>

- Per esigenze implementative che vedremo in seguito, per il nostro Web Service utilizzeremo URI della forma:
 - <http://www.mionegozio.com/ordini/?id=123>

Rappresentazione, URI e metodi di un web service RESTful

- Il quarto passo della nostra analisi ci porta ad individuare quali metodi HTTP possono essere eseguiti sulle singole risorse e con quale significato.
- A questo proposito può essere utile creare una tabella che riassume l'insieme delle operazioni possibili, come nel seguente esempio:

URI	GET	PUT	POST	DELETE
/ordini/?id=xxx	X	X	X	?
/articoli/?id=xxx	X	—	—	—
/fatture/?id=xxx	X	—		

- La prima colonna della tabella contiene i modelli di URI scelti per le nostre risorse, mentre le altre colonne rappresentano i quattro metodi principali del protocollo HTTP. In ciascuna cella corrispondente all'incrocio tra risorsa e metodo abbiamo inserito schematicamente un'informazione che indica l'applicabilità del metodo.
- Ad esempio, con una "X" indichiamo che è possibile leggere, modificare e creare un ordine, mentre l'eliminazione, contrassegnata con "?", dipende da determinate condizioni come lo stato in cui esso si trova (ad esempio, non è possibile eliminare un ordine già evaso).
- Per le altre risorse l'unica operazione possibile è l'acquisizione tramite GET.

La rappresentazione delle risorse

- Sappiamo che REST non stabilisce nessun formato standard per la rappresentazione delle risorse, quindi siamo liberi di utilizzare il formato che ci è più familiare o comodo: dal semplice testo a XML.
- Tuttavia dobbiamo tener presente che un Web Service è pensato per essere utilizzato da un client scritto da altri e molto probabilmente vorremmo avere un utilizzo da parte di un ampio numero e tipo di client.
- Questo ci porta a prendere in considerazione un formato che sia il più possibile condiviso e possibilmente standardizzato, in modo che sia facilitata l'interpretazione e gestione da parte dei vari possibili client.
- Quindi, in linea di massima, se esiste un formato di rappresentazione di risorse universalmente accettato e che può essere utilizzato per i nostri scopi, allora è bene adottarlo.
- Ad esempio, se dobbiamo gestire documenti per cui (X)HTML è sufficientemente espressivo nella rappresentazione, perchè non utilizzarlo?

La rappresentazione delle risorse

- Riallacciandoci al nostro piccolo progetto, nel nostro caso utilizzeremo un formato ad hoc basato su XML. Ad esempio, un ordine potrebbe essere rappresentato come segue:

```
<ordine xmlns="http://schemi.mionegozio.com/ordine">
  <id>123</id>
  <data>01/07/2011</data>
  <cliente>Mario Rossi</cliente>
  <stato>EVASO</stato>
  <dettagli>
    <dettaglio>
      <articolo>Libro</articolo>
      <quantita>1</quantita>
    </dettaglio>
    <dettaglio>
      <articolo>CD</articolo>
      <quantita>5</quantita>
    </dettaglio>
  </dettagli>
</ordine>
```

- Naturalmente questa è una versione molto semplificata di un ordine reale, ma quello che ci interessa in questa fase è dare un'idea di cosa si intende per rappresentazione di una risorsa.

Implementare un Web Service RESTful

- Le considerazioni che abbiamo fatto fin qui sono indipendenti dallo specifico ambiente di programmazione.
- L'unico aspetto che è stato dettato dall'implementazione è stata la scelta del modello di URI che adotteremo per individuare le risorse.
- Lo scopo è rendere chiaro cosa succede dietro le quinte in presenza di una richiesta HTTP sulle nostre risorse e per questo intendiamo gestire queste richieste cercando di non rimanere troppo legati ad un framework particolare.
- D'altronde, il modello REST ci fornisce esclusivamente dei principi lasciandoci liberi da vincoli sulla tecnologia da utilizzare.
- Lo schema URI che utilizziamo è tipo `http://www.mionegozio.com/ordini/?id=123`: il Web server ha il compito di individuare la pagina predefinita nella cartella ordini, tipicamente `default.aspx`, l'identificazione della risorsa specifica la facciamo tramite il parametro `id`.
- Non specificare la pagina predefinita ci permette di mantenere lo schema di URI scelto anche se cambiamo tecnologia di implementazione, utilizzando ad esempio PHP o JSP.
- La pagina che gestisce una risorsa eseguirà del codice analogo al seguente in corrispondenza dell'evento Load per individuare il tipo di azione richiesta dal client.
- In corrispondenza di ciascun metodo HTTP avremo una procedura che si occupa di gestire opportunamente la risorsa.

WEB SERVICES

TEST & DEBUG

Strumenti per il test e il debug di un Web Service

- Una volta creato un Web service occorre metterlo alla prova e verificare se si comporta come ci aspettiamo.
- I motivi di eventuali malfunzionamenti potrebbero essere diversi e di diversa natura.
- Per comprenderne la causa è essenziale intercettare le richieste e le risposte scambiate tra client e Web Service.
- Per testare l'esecuzione del metodo GET su una risorsa sarebbe sufficiente un comune Web browser, ma già per l'esecuzione del metodo POST la cosa si fa un pò più complessa.
- Per gestire i metodi PUT e DELETE dovremmo creare un client ad hoc.
- Possiamo evitare di crearci un client utilizzando uno dei numerosi strumenti per l'esecuzione di richieste HTTP disponibili in rete, che si differenziano più che per le funzionalità di base per la capacità di integrarsi in un ambiente piuttosto che in un altro.

Strumenti per il test e il debug di un Web Service

- Il browser Firefox presenta tutta una serie di add-on molto comodi da usare e con una interfaccia grafica semplice ma abbastanza completa. Come tutti gli strumenti di questo tipo, oltre alla possibilità di richiedere una risorsa con i vari metodi HTTP, è possibile specificare le intestazioni, i parametri, il contenuto del corpo della richiesta e le credenziali di autenticazione.
- Anche RESTClient (<https://addons.mozilla.org/en-US/firefox/addon/restclient/?src=search>) è un add-on di Firefox con funzionalità analoghe a Poster. Tuttavia con lo stesso nome esistono anche altri strumenti per il test ed il debugging di connessioni HTTP. Ad esempio così si chiamano un client Java con versione a riga di comando e versione con interfaccia grafica ed una piccola applicazione scritta in Python
- SoapUI invece è uno strumento desktop un più complesso che fa molto più che da semplice client RESTful (oltre che Soap). Esso è un vero e proprio debugger HTTP con la possibilità di tenere traccia di tutte le connessioni HTTP in uscita da una macchina con possibilità di analisi e tracciamento storico delle richieste senza contare la possibilità di creare un progetto client (un progetto soapUi ovviamente) di test.
- Infine, per chi usa Eclipse, esistono diversi plugin che aggiungono alla funzionalità standard di creazione ed invio di richieste HTTP, alcune caratteristiche tipiche di un ambiente di sviluppo, come ad esempio i suggerimenti automatici quando si digitano parole chiave nelle intestazioni, visualizzazioni diverse del corpo della risposta HTTP e la possibilità di generare il codice di un client RESTful in Java, PHP, C# e in altri linguaggi di programmazione.

Strumenti per il test e il debug di un Web Service

- Al di là dello specifico strumento utilizzato, cosa dobbiamo analizzare per capire qual è la causa di un malfunzionamento? Ecco una breve checklist di punti da tenere presente analizzando le richieste e le risposte scambiate da client e server:
 - Il client riesce ad inviare la richiesta?: Sembra un'osservazione banale, ma può verificarsi che il client non sia in grado di comunicare con il Web service; può essere il caso di un URL scritto male o una regola del firewall che impedisce la comunicazione.
 - La richiesta inviata dal client è conforme alle specifiche del Web Service?: Occorre verificare la richiesta HTTP sia correttamente formulata, analizzando sia l'eventuale contenuto del corpo che le intestazioni HTTP.
 - Il client riesce a ricevere la risposta dal Web Service?: Come per il caso di invio della richiesta, anche per la ricezione potrebbero verificarsi problemi legati al firewall, a meno che la mancata risposta non dipenda da un problema specifico del Web Service.
 - La risposta ricevuta dal client è conforme alle specifiche?: L'analisi della risposta e della sua conformità alle specifiche previste può essere fondamentale per l'individuazione del malfunzionamento; da non trascurare le intestazioni HTTP.

WEB SERVICES

LO STATO DI UN WS REST

Gestire la comunicazione stateless

- Tra i principi su cui si basano le architetture RESTful ce n'è uno abbastanza importante ma spesso poco compreso.
- Si tratta del principio che stabilisce la necessità di avere una comunicazione stateless, senza stato, cioè una comunicazione tra client e server in cui ciascuna richiesta è indipendente dalle altre.
- Una richiesta non deve richiedere al server il recupero di un contesto o stato dell'applicazione.
- Un Web Service o un client REST devono includere nelle intestazioni o nel corpo HTTP tutti i parametri, contesto e dati necessari al server per generare una risposta.
- Una comunicazione senza stato consente di ottimizzare le prestazioni del Web Service e di semplificare la progettazione e l'implementazione dei componenti lato server, perchè l'assenza della gestione dello stato rimuove la necessità di sincronizzare dati di sessione con un'applicazione esterna.
- Inoltre l'assenza di stato nella comunicazione offre la possibilità di scalare l'applicazione su più server per la distribuzione del carico di lavoro o per fault tolerance.

Gestire la comunicazione stateless

- Il protocollo HTTP è intrinsecamente senza stato, ma molto spesso nello sviluppo di applicazioni Web vengono artificialmente ricreate le condizioni per una comunicazione con stato.
- Ad esempio, l'uso di chiavi di sessione, in diversi framework di sviluppo per il Web, non fa altro che introdurre lo stato della comunicazione al di sopra del protocollo HTTP, in genere sfruttando i cookie.
- In una architettura RESTful l'uso di comunicazioni con stato sarebbe vietato: il server non dovrebbe tenere traccia delle relazioni tra le diverse richieste. Se è necessario gestire uno stato della comunicazione, questo è compito del client.

Stato delle risorse e dell'applicazione

- Il fatto che i principi REST escludano la gestione dello stato della comunicazione non deve però far pensare che i Web Service RESTful siano senza stato.
- L'acronimo REST sta per REpresentational State Transfer, sottolineando proprio la centralità della gestione dello stato in un sistema distribuito.
- Lo stato che REST prende in considerazione è però quello delle risorse e dell'intera applicazione.
- Lo stato delle risorse è dato dall'insieme dei valori delle caratteristiche di una risorsa in un dato momento.
- Un Web Service è responsabile della gestione dello stato delle risorse.
- Un client può accedere allo stato di una risorsa tramite le diverse rappresentazioni della risorsa stessa e contribuire a modificarlo per mezzo dei metodi PUT, POST e DELETE dell'HTTP.

Stato delle risorse e dell'applicazione

- Lo stato del client è rappresentato dall'insieme del contesto e delle risorse ottenute in uno specifico momento.
- Il sever può influenzare le transizioni di stato del client inviando differenti rappresentazioni in risposta alle sue richieste.
- Lo stato dell'applicazione, cioè del risultato dell'interazione tra client e server, è dato dallo stato del client e delle risorse gestite dal server.
- Lo stato dell'applicazione determina le modalità di modifica dello stato delle risorse e del client.
- A differenza di quanto avviene in buona parte delle applicazioni Web, dove lo stato dell'applicazione viene spesso mantenuto dal server insieme allo stato della comunicazione, lo stato dell'applicazione in un'architettura RESTful è il frutto della collaborazione di client e server, ciascuno con i propri ruoli e responsabilità.
- Nella progettazione di un Web Service RESTful questa separazione dei ruoli deve essere chiara e spesso occorre ripensare opportunamente la gestione dello stato in termini di gestione di risorse.

Stato delle risorse e dell'applicazione

- Chiariamo questo concetto con un esempio: come viene gestito di solito un carrello in una comune applicazione di e-commerce?
- Tipicamente si ricorre ad una rappresentazione lato server associata alla sessione corrente dell'utente.
- Per quanto abbiamo detto finora, questo approccio non è RESTful, dal momento che richiede il mantenimento dello stato della sessione.
- Per un approccio RESTful abbiamo a disposizione due possibili soluzioni: demandare al client il compito di gestire il carrello o gestire il carrello sul server come una risorsa.
- Nel primo caso il client avrà una struttura dati in cui terrà traccia degli articoli a cui l'utente è interessato.
- Nel momento della generazione della richiesta di un nuovo ordine da inviare al server, gli articoli annotati nel carrello verranno riportati nell'ordine.

Stato delle risorse e dell'applicazione

- In alternativa è possibile che il nostro Web Service preveda una risorsa carrello dedicata al mantenimento degli articoli scelti dall'utente durante la fase di acquisto.
- È da sottolineare il fatto che il carrello è una risorsa come le altre e non è associata alla sessione corrente.
- È quindi accessibile tramite URI in qualsiasi momento, è persistente ed è gestibile tramite i metodi HTTP.
- La trasformazione in risorse di elementi tradizionalmente gestiti tramite lo stato della sessione è un approccio da tenere presente durante la progettazione di un Web Service RESTful.

Transizioni di stato, hypermedia e il principio HATEOAS

- Tornando ancora una volta all'acronimo REST, REpresentational State Transfer, notiamo che in esso viene fatto riferimento al trasferimento di stato, cioè al fatto che un'applicazione passi da uno stato all'altro.
- Uno dei principi REST suggerisce l'uso di collegamenti tra risorse come modalità di transizione da uno stato all'altro.
- Questo principio è noto anche con l'acronimo HATEOAS, Hypermedia As The Engine Of Application State, e focalizza l'attenzione sull'utilizzo dei collegamenti ipertestuali, anzi ipermediali, come meccanismo di base per far passare un'applicazione da uno stato ad un altro.
- Nella visione REST, l'esecuzione di un'applicazione può essere rappresentata da una rete di risorse in cui un client naviga seguendo i collegamenti ammissibili tra una risorsa e l'altra.
- Questa visione probabilmente ci suonerà abbastanza familiare, dal momento che la mettiamo in pratica tutti i giorni navigando tra le pagine del Web.

Transizioni di stato, hypermedia e il principio HATEOAS

- Concettualmente corrisponde ad una macchina a stati finiti in cui gli stati sono rappresentati dallo stato delle singole risorse e le transizioni da uno stato all'altro sono attivate dai collegamenti ipermediali.
- Il principio che sancisce il legame tra transizioni di stato e collegamenti tra risorse è purtroppo scarsamente sfruttato nelle attuali implementazioni di Web service di tipo RESTful.
- La maggior parte di queste implementazioni si limita a definire le rappresentazioni delle risorse e le modalità di interazione tramite i metodi HTTP.
- Se il risultato di una interazione con il server è un contenitore di risorse, allora all'interno della risorsa abbiamo gli URI delle risorse contenute, altrimenti nella maggior parte dei casi non si hanno altri collegamenti.
- Non possiamo dire che un Web Service di questo tipo non sia RESTful, ma nella maggior parte dei casi non sfrutta a pieno le potenzialità dei principi REST.
- Il principio HATEOAS, infatti, intende incoraggiare non solo l'uso di collegamenti per rappresentare risorse composte, ma anche per definire qualsiasi altra relazione tra le risorse e per controllare le transizioni ammissibili tra uno stato e l'altro dell'applicazione.

Transizioni di stato, hypermedia e il principio HATEOAS

- Per fare un esempio, consideriamo la seguente rappresentazione di un ordine nel nostro ipotetico negozio online :

```
<ordine xmlns="http://schemi.mionegozio.com/ordine">
  <id>123</id>
  <data>01/07/2011</data>
  <cliente>Mario Rossi</cliente>
  <stato>EVASO</stato>
  <dettagli>
    <dettaglio>
      <articolo>Libro</articolo>
      <quantita>1</quantita>
    </dettaglio>
    <dettaglio>
      <articolo>CD</articolo>
      <quantita>5</quantita>
    </dettaglio>
  </dettagli>
</ordine>
```

- In esso non è riportato alcun collegamento con altre risorse gestite dal sistema; come può un client conoscere la fattura associata a quest'ordine? Dovrà effettuare un'apposita richiesta al Web Service, ma dovrà conoscere l'URI della risorsa.
- Le specifiche del particolare Web Service potrebbero spiegare (allo sviluppatore) come fare ad ottenere la fattura di un determinato ordine.

Transizioni di stato, hypermedia e il principio HATEOAS

- la cosa più naturale nell'ottica del principio HATEOAS sarebbe di includere un collegamento direttamente nella rappresentazione dell'ordine:

```
<ordine xmlns="http://schemi.mionegozio.com/ordine">
  <id>123</id>
  <data>01/07/2011</data>
  <cliente>Mario Rossi</cliente>
  <stato>EVASO</stato>
  <dettagli>
    <dettaglio>
      <articolo>Libro</articolo>
      <quantita>1</quantita>
    </dettaglio>
    <dettaglio>
      <articolo>CD</articolo>
      <quantita>5</quantita>
    </dettaglio>
  </dettagli>
  <link rel="fattura" mediaType="application/xml"
        href="http://www.mionegozio.com/fatture/?id=789" />
</ordine>
```


Transizioni di stato, hypermedia e il principio HATEOAS

- In questo modo il client ha già nella rappresentazione dell'ordine tutte le informazioni necessarie per accedere alla fattura associata all'ordine.
- Tra l'altro, la presenza o meno del collegamento alla fattura associata all'ordine fornisce implicitamente un'altra informazione al client: se il collegamento non è presente nella rappresentazione dell'ordine vuol dire che la fattura non è ancora stata emessa, se invece è presente vuol dire che è stata emessa ed è accessibile.
- Questa consente un maggior controllo della transizione da uno stato all'altro dell'applicazione, evitando transizioni non ammissibili in un dato momento.
- Sfruttando pienamente il principio HATEOAS è possibile creare servizi Web con scarso accoppiamento tra client e server.
- Infatti, se il server riorganizza le relazioni tra le risorse, il client è in grado di trovare tutto ciò che serve nelle rappresentazioni ricevute.
- Potenzialmente tutto quello che servirebbe ad un client è solo l'URI della risorsa iniziale, ad esempio quello dell'ordine.
- Come avanzare tra uno stato e l'altro dell'applicazione verrà indicato man mano che si seguono i collegamenti incorporati nelle successive rappresentazioni di risorse.

WEB SERVICES

REST e sicurezza HTTP

REST e sicurezza HTTP

- Uno degli aspetti cruciali di qualsiasi applicazione Web è la gestione della sicurezza. L'esposizione di risorse accessibili tramite i meccanismi tipici del Web, come avviene con l'approccio REST per la creazione di Web Service, non si sottrae a questa necessità.
- In linea di massima la sicurezza di un sistema coinvolge almeno i seguenti aspetti:
 - l'autenticazione, cioè la capacità di identificare le parti coinvolte in una comunicazione
 - l'autorizzazione, cioè la concessione dei diritti di accesso ad una risorsa in base all'identità
 - la fiducia, cioè la capacità di confidare nel risultato di un'operazione, come ad esempio l'autenticazione o l'autorizzazione, eseguita da terze parti
 - la riservatezza, cioè la capacità di mantenere l'informazione privata durante la trasmissione o la memorizzazione
 - l'integrità, cioè la capacità di prevenire che l'informazione possa essere modificata da terzi

REST e sicurezza HTTP

- fortunatamente questi aspetti sono stati efficacemente affrontati in ambito Web ormai da diverso tempo per poter affermare di essere giunti ad un modello abbastanza maturo.
- In effetti, dal momento che l'approccio REST nella creazione di Web Service è largamente basato sul protocollo HTTP, la gestione della sicurezza ne eredita le caratteristiche ormai collaudate.
- In sostanza, per gestire l'identità nell'ambito di un Web Service RESTful possono essere utilizzati i meccanismi propri del protocollo HTTP, come ad esempio l'HTTP Basic Authentication o l'HTTP Digest Authentication.
- Ad esempio, se un client richiede l'accesso ad una risorsa protetta, il server può richiedere l'autenticazione con una risposta HTTP analoga alla seguente:

```
401 Unauthorized
WWW-Authenticate:
Basic realm="risorsaRiservata"
```

- La risposta richiede di autenticarsi utilizzando l'HTTP Basic Authentication. Il client invierà una risposta di questo tipo:
GET /ordini/?123 HTTP/1.1 Host: www.mionegozio.com Authorization: Basic QWxhZGRpbjpvGVuIHNLc2FtZQ==
- dove le credenziali di autenticazione vengono passate sotto forma di stringa codificata in Base64.

REST e sicurezza HTTP

- Da questo momento in poi, ciascuna richiesta inviata al server conterrà le credenziali prevenendo ulteriori richieste da parte del server e mantenendo la natura stateless dell comunicazione HTTP.
- Naturalmente l'invio delle credenziali con la semplice codifica Base64 non è sufficiente garanzia di riservatezza.
- L'uso dell'HTTP Digest Authentication è sicuramente un passo avanti per evitare la trasmissione in chiaro della password, ma non è esente da possibili attacchi.
- La soluzione che in genere andrebbe adottata consiste nella creazione di un canale di trasmissione sicuro, come può essere HTTPS, cioè HTTP over SSL/TLS.
- L'adozione di HTTPS garantisce la riservatezza e l'integrità nella trasmissione delle informazioni, coprendo gli altri aspetti di sicurezza.

Sicurezza con “sessioni REST” e tramite servizi di terze parti

- Nel caso in cui i meccanismi di sicurezza propri del protocollo HTTP non siano sufficienti o adeguati per il tipo di Web Service da sviluppare, è sempre possibile gestire la sicurezza in modo personalizzato.
- In genere questo accade per la gestione dell'autenticazione e dell'autorizzazione dei client, dove i criteri possono essere diversi in base allo specifico dominio del problema.
- Non c'è un obbligo nell'utilizzare i meccanismi standard del protocollo HTTP, ma è bene sottolineare che ogni eventuale soluzione personalizzata deve rispettare i principi REST.
- Il rischio di cadere in una soluzione non-RESTful per gestire l'autenticazione e l'autorizzazione dei client è alto, dal momento che un istinto quasi involontario tenterebbe di re-introdurre il concetto di sessione associata al client appena autenticato.
- In realtà il problema non sta nel concetto di sessione, ma nella modalità di gestione.
- Dal momento che REST non ammette il mantenimento dello stato della sessione tra una richiesta e le altre, occorre che client e server si scambino tutta l'informazione necessaria per ricreare la sessione ad ogni interazione.
- Questo vuol dire rigenerare la sessione ad ogni richiesta utilizzando le credenziali fornite dal client, con potenziale perdita di prestazioni anche se i criteri di autenticazione ed autorizzazione non sono molto complessi.

Lo stato come risorsa

- Un'alternativa potrebbe essere trasformare la sessione in risorsa. In pratica si tratta dello stesso approccio adottato per l'implementazione del carrello del nostro negozio online.
- L'autenticazione di un client corrisponde alla richiesta di creazione di una risorsa di tipo sessione: se le credenziali fornite dal client sono valide, allora viene creata la risorsa ed inviato il corrispondente l'URI al client, che da questo momento in poi invierà al server con ogni richiesta.
- Ad ogni richiesta il server recupererà la sessione per effettuare le necessarie verifiche e in caso positivo soddisferà le richieste del client.
- Il server avrà la responsabilità di gestire opportunamente la sessione, come ad esempio eliminarla dopo un determinato periodo di inattività, ma non la manterrà in memoria, la gestirà come una normale risorsa persistente.

OpenID e OAuth: gestione esterna della sicurezza

- utropo la gestione dell'autenticazione e dell'autorizzazione dei client rischia di complicare la progettazione e l'implementazione di un Web Service.
- La soluzione ideale sarebbe di delegare ad un servizio esterno il compito di gestire i client: in pratica un Web Service specializzato nella gestione dell'autenticazione e/o dell'autorizzazione.
- Negli ultimi tempi sono venuti alla ribalta servizi aperti che si pongono proprio questo obiettivo: decentralizzare la gestione di alcuni aspetti della sicurezza.
- Tra i servizi di questo tipo più noti e diffusi nell'ambito della realizzazione di Web Service RESTful ci sono OpenID (<https://openid.net/>) e OAuth (<https://oauth.net/>).
- È naturale che l'utilizzo di servizi esterni per la gestione di alcuni aspetti della sicurezza presuppone che il Web Service riponga piena fiducia nel fornitore del servizio, senza la quale questo tipo di servizi non avrebbero senso.

OpenID

- OpenID è un protocollo per la gestione dell'identità online.
- Esso consente ad un client di presentare ad un Web Service o ad un sito Web un URI come dichiarazione della propria identità.
- L'applicazione Web chiede conferma della validità al relativo servizio di autenticazione (OpenID provider) il quale, dopo aver interagito con il client, conferma o smentisce l'autenticità dell'identità dichiarata.

OAuth

- OAuth è un protocollo per la concessione a terze parti di autorizzazioni per l'accesso a risorse protette.
- In pratica, un client può consentire ad un Web Service l'accesso ad una risorsa protetta, ospitata da un service provider, senza fornire le proprie credenziali.

WEB SERVICES

Framework per servizi RESTful

Framework per servizi RESTful

- L'esempio di implementazione di Web Service che abbiamo visto fin'ora non faceva uso di framework. Ci siamo basati sulla gestione diretta delle richieste e delle risposte HTTP e della rappresentazione delle risorse.
- In altre parole, il Web Service, oltre a gestire la logica del servizio vero e proprio, si occupava anche dei dettagli relativi alla comunicazione con il client.
- La scelta era di taglio didattico, dal momento che l'obiettivo era di mostrare come funziona l'interazione tra un client ed un servizio RESTful nei suoi meccanismi essenziali, senza alcuna infrastruttura aggiuntiva.
- Sebbene REST non indichi l'utilizzo di framework o librerie specifiche, in una situazione reale l'uso di un framework può semplificare l'implementazione di un Web Service, permettendoci di concentrarci sulla logica applicativa senza pensare ai dettagli relativi alla presentazione del servizio ed alla comunicazione con il client.

Le caratteristiche di un framework per servizi RESTful

- Generalmente un framework per la creazione di Web Service RESTful mette a disposizione alcune caratteristiche fondamentali, vediamo quali.
 - Associazione di URI alle risorse: Questa caratteristica, chiamata anche URI routing, consente di associare uno o più URI ad una risorsa, favorendo l'uso di URI con schema posizionale; nel caso del nostro Web Service, ad esempio, un framework ci avrebbe consentito di utilizzare URI del tipo <http://www.mionegozio.com/ordini/1234> al posto di <http://www.mionegozio.com/ordini/?id=1234> ; oltre che per una questione estetica (ricordiamo che i due URI sono perfettamente equivalenti da un punto di vista REST), la capacità di associare un URI ad una risorsa svincola in genere dalla corrispondenza uno a uno tra posizione fisica del modulo che gestisce la risorsa ed il suo URI. In altre parole, non siamo obbligati a pubblicare la pagina che gestisce la rappresentazione di un ordine nella cartella (virtuale o fisica) /ordini.
 - Gestione della comunicazione HTTP: Un framework REST si occupa dei dettagli della comunicazione tra client e server, separando di fatto la logica di gestione della risorsa dalla comunicazione. Non dovremo accedere direttamente alle richieste HTTP, analizzarle, mappare la richiesta su una procedura interna e creare una risposta manualmente. Quello che in linea di massima un framework richiede di fare è specificare il codice da eseguire in corrispondenza a ciascun metodo HTTP.

Le caratteristiche di un framework per servizi RESTful

- Semplificazione della rappresentazione delle risorse: La maggior parte dei framework RESTful prevedono il supporto di formati predefiniti, come ad esempio Atom o JSON, cioè sono in grado di creare rappresentazioni di risorse in questi formati. Quasi tutti i framework consentono però anche la generazione della rappresentazione in formati personalizzati. Alcuni framework sono anche in grado di gestire in automatico della cosiddetta content negotiation, cioè la fornitura di una rappresentazione nel formato richiesto dal client.
- Funzionalità di supporto: Un framework può anche fornire supporto per la semplificazione di alcune funzionalità come la gestione dichiarativa della cache, dell'autenticazione, degli errori o l'interfacciamento a protocolli esterni, come ad esempio OAuth.
- Supporto per i client: Anche se la realizzazione di un client RESTful è in linea di massima indipendente dal server, alcuni framework mettono a disposizione un supporto per la creazione di un client, semplificando ad esempio la gestione delle chiamate e risposte HTTP e l'interpretazione delle rappresentazioni delle risorse.

Quale Framework scegliere?

- Esistono framework per lo sviluppo di Web Service RESTful per i diversi ambienti di programmazione.
- Nelle prossime slides riportiamo una breve sintesi delle caratteristiche di alcuni tra i più diffusi raggruppandoli per ambiente di programmazione.

WEB SERVICES

.Net

ASP.Net MVC & ASP.Net Web Api

- .Net supporta una serie di librerie native per la realizzazione di servizi RESTFul poggianti dal punto di vista architetturale sulla tecnologia ASP.Net MVC; Tali librerie vanno sotto il nome di ASP.Net Web Api
- E' strettamente legato al protocollo HTTP
- Usa il protocollo di base e i formati quali HTTP, WebSockets, SSL, JSON e XML. Non è disponibile alcun supporto per i protocolli di livello superiore, ad esempio la messaggistica affidabile o le transazioni.

ASP.Net MVC & ASP.Net Web Api

- .Net supporta una serie di librerie native per la realizzazione di servizi RESTFul poggianti dal punto di vista architetturale sulla tecnologia ASP.Net MVC; Tali librerie vanno sotto il nome di ASP.Net Web Api
- E' strettamente legato al protocollo HTTP
- Usa il protocollo di base e i formati quali HTTP, WebSockets, SSL, JSON e XML. Non è disponibile alcun supporto per i protocolli di livello superiore, ad esempio la messaggistica affidabile o le transazioni.
- Documentazione all'URL: <https://www.asp.net/web-api>

- WCF è parte integrante del .NET Framework e fornisce un modello di programmazione per la realizzazione di applicazioni service-oriented.
- WCF è il modello di programmazione unificato di Microsoft per la compilazione di applicazioni orientate ai servizi.
- Consente agli sviluppatori di compilare soluzioni transazionali protette e affidabili in grado di integrarsi a piattaforme diverse e interagire con investimenti esistenti.
- Abilita i servizi di compilazione che supportano più protocolli di trasporto (HTTP, TCP, UDP e trasporti personalizzati) e consentono il passaggio da un protocollo all'altro.
- Abilita i servizi di compilazione che supportano più codifiche (testo, MTOM e binario) dello stesso tipo di messaggio e consente il passaggio da una codifica all'altra.
- Supporta i servizi di compilazione con gli standard WS-* come la messaggistica affidabile, le transazioni, la sicurezza dei messaggi.
- E' più orientato alla realizzazione di servizi SOAP; i SOAP WCF possono essere descritti in WSDL per permettere agli strumenti automatizzati di generare i proxy client anche per i servizi con schemi complessi.

OpenRasta

- OpenRasta è un progetto Open Source per la creazione di Web Service ed applicazioni in stile REST.
- Nella sua implementazione adotta un pattern architetturale simile a Model View Controller, chiamato Resource Codec Handler. In pratica il framework intende separare la dipendenza tra una risorsa, la sua rappresentazione gestita da un codec e la sua gestione effettuata da un handler.
- È in grado di generare rappresentazioni in XML, JSON e HTML tramite WebForms, ma grazie alla sua architettura è possibile creare nuovi codec senza particolare difficoltà.
- Per quanto riguarda la sicurezza, supporta Digest HTTP Authentication senza necessità di appoggiarsi ad una Active Directory.
- URL: <http://openrasta.org/>

Service Stack

- Anche Service Stack è un progetto Open Source pensato per .NET e Mono ed utilizzabile sia per applicazioni ASP.NET che per applicazioni di tipo Windows service o console. La velocità di esecuzione è uno degli obiettivi del progetto e viene molto enfatizzata ad esempio nella generazione di rappresentazioni di risorse in XML, JSON, HTML e CSV.
- Oltre all'implementazione di Web Service in stile REST, supporta anche Web Service basati su SOAPed RPC.
- Come per WCF Starter Kit, anche Service Stack è in grado di produrre automaticamente pagine Web di documentazione del Web Service.
- URL: <https://servicestack.net/>

WEB SERVICES

JAVA

RESTFul in Java

- Java è legato fortemente al mondo Open Source motivo per cui vi sono miriadi di soluzioni possibili utili per l'implementazione di un Web-Service RESTFul.
- Di seguito riportiamo alcuni framework corrispondenti alle soluzioni native piuttosto che a quelle maggiormente utilizzate

- E' possibile realizzare servizi RESTFul in Java sfruttando le librerie J2EE native inerenti a JSP e JSF
- Sfruttando il pattern MVC e progettando ad hoc Servlet e View è possibile costruire qualsivoglia servizio RestFul

Spring MVC

- Spring è un framework open source per lo sviluppo di applicazioni su piattaforma Java.
- A questo framework sono associati tanti altri progetti, che hanno nomi composti come Spring Boot, Spring Data, Spring Batch, eccetera. Questi progetti sono stati ideati per fornire funzionalità aggiuntive al framework.
- Segnaliamo Spring Tool Suite corrispondente a un ambiente di sviluppo integrato, derivato da Eclipse, che permette uno sviluppo più veloce di applicativi basati su Spring. Fornisce supporto al linguaggio Java, al framework Spring ed all'eventuale ambiente di sviluppo.

Jersey

- Jersey è un framework Open Source che implementa le specifiche JAX-RS, Java API for RESTful Web Services, utilizzando le annotazioni per mappare una classe Java ad una risorsa Web.
- All'implementazione di JAX-RS, Jersey aggiunge nuove API per consentirne l'estensione secondo le necessità dello sviluppatore. Supporta nativamente rappresentazioni in Atom e JSON il protocollo OAuth, sia come client che come server.
- Jersey si integra in Spring e Guice e fornisce funzionalità per la realizzazione di client RESTful.
- URL: <https://jersey.github.io/>

- RETEasy è l'implementazione di JBoss delle specifiche JAX-RS. Esso consente non solo di realizzare Web Service RESTful, ma più in generale applicazioni Java ispirate ai principi REST.
- Oltre che ad essere integrato in JBoss, RETEasy può essere eseguito in un qualsiasi contenitore servlet. Supporta rappresentazioni in XML, JSON, YAML, Atom e si integra in EJB, Spring e Guice.
- URL: <https://reteasy.github.io/>

- Restlet è un framework Open Source che supporta una visione di API REST indipendente dal ruolo di client e server.
- Il framework è identico sia per lo sviluppo di Web service che di client RESTful e prevede connettori che gli consentono di integrarsi in diversi ambienti di esecuzione, come ad esempio Spring, JAXB, ma anche Apache HTTP Client, JavaMail, JDBC.
- Prevede rappresentazioni in XML, JSON, Atom. e supporta Basic e Digest HTTP Authentication, Amazon S3 Authentication, Google SDC (Secure Data Connector), OAuth ed altri meccanismi di sicurezza.
- URL: <https://restlet.com/>

Play Framework

- Play Framework semplifica la creazione, gestione e l'implementazione di applicazioni Web con Java & Scala.
- È costruito su Akka , basato su un'architettura leggera e senza stato. Dovrebbe essere utilizzato per applicazioni altamente scalabili che richiedono un consumo minimo di risorse di CPU e memoria.
- URL: <https://www.playframework.com/>